

ENUNCIADO TAREA 2

ALGORITMOS Y COMPLEJIDAD

«AniMaraton»

Fecha límite de entrega: **24 de junio de 2026**

Equipo ALGORITMOS Y COMPLEJIDAD 2026-1

12 de mayo de 2026

21:21

Versión 0.0

Índice

1. Objetivos	2
1.1. Problema	2
1.2. Entrada y salida	3
2. Especificaciones	5
2.1. Implementaciones	5
3. Condiciones de entrega	7
A. Casos de prueba	8

1. Objetivos

El objetivo de esta tarea 2 es comparar diferentes técnicas de resolución de un mismo problema de optimización, evaluando su **tiempo de ejecución, uso de memoria y calidad de solución**.

Deberán implementar tres enfoques: un **algoritmo de fuerza bruta**, dos **heurísticas greedy subóptimas** (2 algoritmos distintos que no garantizan la solución óptima) y un **algoritmo de programación dinámica**. Además, se analizará el rendimiento y la calidad de las soluciones para determinar qué tan cercanas están al resultado óptimo.

1.1. Problema

En **AniMarathon**, una plataforma de streaming está por terminar una promoción de acceso gratuito. Usted quiere aprovechar las últimas horas disponibles para ver la mayor cantidad de contenido valioso posible antes de que expire su suscripción.

La plataforma tiene distintos animes disponibles. Cada anime está compuesto por una cantidad determinada de capítulos. Cada capítulo tiene una duración en minutos, un costo de energía mental y un puntaje de satisfacción. Además, si una persona logra terminar todos los capítulos de un anime, recibe un bono adicional de satisfacción por completar la historia.

Sin embargo, no se permite ver capítulos aislados de una serie sin haber visto los anteriores. Es decir, para cada anime se puede ver un prefijo de capítulos: ninguno, solo el primer capítulo, los dos primeros capítulos, los tres primeros capítulos, y así sucesivamente. Si se ve el capítulo j de un anime, entonces necesariamente se deben haber visto todos los capítulos anteriores de ese mismo anime.

Usted dispone de una cantidad máxima de minutos M y una cantidad máxima de energía E . El objetivo es decidir cuántos capítulos ver de cada anime para maximizar la satisfacción total, sin exceder el tiempo ni la energía disponibles.

Cada anime i tiene q_i capítulos y un bono de completación b_i . Cada capítulo j del anime i se representa mediante:

$$(t_{i,j}, c_{i,j}, v_{i,j})$$

donde:

- $t_{i,j}$ es la duración en minutos del capítulo j del anime i .
- $c_{i,j}$ es el costo de energía del capítulo j del anime i .
- $v_{i,j}$ es el puntaje de satisfacción del capítulo j del anime i .

Para cada anime i , se debe escoger un valor k_i tal que:

$$0 \leq k_i \leq q_i$$

donde k_i representa la cantidad de capítulos consecutivos vistos desde el inicio del anime i .

La satisfacción obtenida por el anime i se define como:

$$\sum_{j=1}^{k_i} v_{i,j}$$

y, si $k_i = q_i$, se agrega además el bono de completación b_i .

El objetivo es maximizar la satisfacción total:

$$\text{MaxSatisfaccion} = \sum_{i=1}^n \left(\sum_{j=1}^{k_i} v_{i,j} + B_i(k_i) \right)$$

donde:

$$B_i(k_i) = \begin{cases} b_i & \text{si } k_i = q_i \\ 0 & \text{en otro caso} \end{cases}$$

sujeto a las restricciones:

$$\sum_{i=1}^n \sum_{j=1}^{k_i} t_{i,j} \leq M$$

$$\sum_{i=1}^n \sum_{j=1}^{k_i} c_{i,j} \leq E$$

Dominio de los valores y casos de prueba

Para esta tarea, los valores de los parámetros cumplen las siguientes restricciones:

- n , la cantidad de animes disponibles, es un entero tal que $1 \leq n \leq 200$.
- q_i , la cantidad de capítulos del anime i , es un entero tal que $1 \leq q_i \leq 30$.
- La cantidad total de capítulos, $Q = \sum_{i=1}^n q_i$, cumple $1 \leq Q \leq 700$.
- M , la cantidad máxima de minutos disponibles, cumple $1 \leq M \leq 3000$.
- E , la energía máxima disponible, cumple $1 \leq E \leq 500$.
- $t_{i,j}$, la duración de cada capítulo, cumple $1 \leq t_{i,j} \leq 300$.
- $c_{i,j}$, el costo de energía de cada capítulo, cumple $1 \leq c_{i,j} \leq 100$.
- $v_{i,j}$, la satisfacción de cada capítulo, cumple $1 \leq v_{i,j} \leq 10^9$.
- b_i , el bono por completar el anime i , cumple $0 \leq b_i \leq 10^9$.
- Los nombres de los animes son strings formados por letras minúsculas, números o guiones bajos, sin espacios.

1.2. Entrada y salida

Formato de entrada

El input se presenta de la siguiente manera:

```
n M E
anime_1 q_1 b_1
t_1_1 c_1_1 v_1_1
t_1_2 c_1_2 v_1_2
...
t_1_q1 c_1_q1 v_1_q1
anime_2 q_2 b_2
t_2_1 c_2_1 v_2_1
...
anime_n q_n b_n
t_n_1 c_n_1 v_n_1
...
t_n_qn c_n_qn v_n_qn
```

Donde:

- n es la cantidad de animes disponibles.
- M es la cantidad máxima de minutos disponibles.
- E es la energía máxima disponible.
- q_i es la cantidad de capítulos del anime i .
- b_i es el bono de satisfacción por completar el anime i .
- Cada capítulo entrega su duración, costo de energía y satisfacción.

Formato de salida

MaxSatisfaccion

donde MaxSatisfaccion es la máxima satisfacción total que se puede obtener respetando las restricciones del problema.

Ejemplo de entrada

```
4 240 90
shonen_quest 3 25
45 18 40
50 22 45
55 25 60
romcom_days 2 15
30 10 25
35 12 30
mecha_nova 2 50
60 30 65
70 35 75
slice_cafe 1 5
25 8 18
```

Ejemplo de salida

260

Explicación

Una selección óptima consiste en:

- Ver completo mecha_nova:

$$60 + 70 = 130 \text{ minutos}$$

$$30 + 35 = 65 \text{ energía}$$

$$65 + 75 + 50 = 190 \text{ satisfacción}$$

- Ver completo romcom_days:

$$30 + 35 = 65 \text{ minutos}$$

$$10 + 12 = 22 \text{ energía}$$

$$25 + 30 + 15 = 70 \text{ satisfacción}$$

El uso total de recursos es:

$$130 + 65 = 195 \leq 240$$

$$65 + 22 = 87 \leq 90$$

La satisfacción total obtenida es:

$$190 + 70 = 260$$

Por lo tanto, la respuesta correcta para este caso es:

$$\text{MaxSatisfaccion} = 260$$

2. Especificaciones

En esta sección se describen los pasos a seguir para completar la tarea, la cual consiste en desarrollar código en C++ «[Implementaciones](#)» y en realizar un informe. Se espera que cada uno de los pasos se realice de manera ordenada y siguiendo las instrucciones dadas.

Abra este documento en algún lector de PDF que permita hipervínculos, ya que en este documento el texto en color [azul](#) suele indicar un hipervínculo.

2.1. Implementaciones

(1) Implementar cada uno de los algoritmos en C++:

- **Algoritmo Fuerza Bruta:** Resolver el problema mediante un enfoque exhaustivo que permita obtener una solución óptima para instancias pequeñas.
Archivo: `code/implementation/algorithms/brute-force.cpp`.
- **Algoritmos Greedy:** Implementar dos heurísticas greedy (*subóptimas*) para el problema. Estas heurísticas deben construir una selección válida de capítulos, aunque no necesariamente óptima.
Archivos: `code/implementation/algorithms/greedy{1,2}.cpp`.
Cada heurística greedy debe utilizar un criterio local distinto. Dicho criterio debe estar claramente descrito y justificado en el informe.
- **Programación Dinámica:** Implementar una solución óptima para el problema mediante programación dinámica.
Archivo: `code/implementation/algorithms/dynamic-programming.cpp`.

(2) **Restricciones de validez de una solución:**

Una solución entregada por cualquier algoritmo debe cumplir las siguientes condiciones:

- Para cada anime, sólo se puede seleccionar un prefijo de capítulos.
- No se permite seleccionar un capítulo si no fueron seleccionados todos los capítulos anteriores del mismo anime.
- La suma total de minutos utilizados no puede exceder M .
- La suma total de energía utilizada no puede exceder E .
- El bono de completación de un anime se suma únicamente si se seleccionan todos sus capítulos.

(3) **Mediciones de tiempo y memoria:** Implementar el programa principal en C++ `code/implementation/general.cpp` y su respectivo `makefile` que ejecute los algoritmos y genere archivos de salida en los directorios:

- `code/implementation/data/measurements/`
- `code/implementation/data/outputs/`

Los archivos generados deben permitir comparar el tiempo de ejecución, el uso de memoria y el resultado obtenido por cada algoritmo.

(4) **Generación de gráficos:** Implementar scripts en Python que lean los archivos de medición y generen gráficos en formato PNG en:

- `code/implementation/data/plots/`
- Scripts de ejemplo: `code/implementation/scripts/testcases_generator.py` `pyplot_generator.py`.

Se espera generar gráficos que permitan analizar:

- Tiempo de ejecución según tamaño de entrada.
- Uso de memoria según tamaño de entrada.
- Calidad de solución de los algoritmos greedy respecto de la solución óptima.

- Comportamiento de los algoritmos al variar la cantidad de animes, capítulos, minutos disponibles y energía disponible.

(5) **Documentación:** Documentar cada uno de los pasos anteriores:

- Completar el archivo `README.md` del directorio `code`.
- Documentar en cada uno de sus programas, al inicio de cada archivo, fuentes de información, referencias y bibliografía utilizada.
- Indicar claramente cómo compilar, ejecutar y generar los casos de prueba.

Generar un informe en $\text{\LaTeX}$ que contenga los resultados obtenidos y una discusión sobre ellos.

- No se debe modificar la estructura del informe.
- Las indicaciones se encuentran en el archivo `README.md` del repositorio y en la plantilla de $\text{\LaTeX}$.
- El informe final compilado (`reporte.pdf`) debe generarse dentro de la carpeta `report/`.

El informe debe incluir, al menos, los siguientes puntos:

- Explicación del algoritmo de fuerza bruta.
- Explicación de las dos heurísticas greedy implementadas.
- Explicación de la solución mediante programación dinámica.
- Análisis de complejidad temporal y espacial de cada enfoque.
- Comparación experimental de tiempo y memoria.
- Comparación de calidad de solución entre greedy y programación dinámica.
- Conclusiones sobre el comportamiento de los algoritmos.

3. Condiciones de entrega

- (1) La tarea es individual, sin excepciones.
- (2) La entrega debe realizarse vía `aula.usm.cl`, subiendo un archivo `.zip` que contenga todo el contenido de su proyecto (código, informe, datos, gráficos, etc.).
- (3) Si se utiliza algún código, idea, algoritmo, librería, explicación o contenido extraído de otra fuente, este debe ser citado en el lugar exacto donde se utilice.
- (4) Asegúrese de que todas sus entregas tengan sus datos completos: número de la tarea, ramo, semestre, nombre y rol. Puede incluirlos como comentarios en sus fuentes $\LaTeX$ (en $\TeX$ los comentarios son desde `%` hasta el final de la línea) o en posibles programas. Anótese como autor de los textos.
- (5) Si usa material adicional al discutido en clases, detállelo. Agregue información suficiente para ubicar ese material (en caso de no tratarse de discusiones con compañeros de curso u otras personas).
- (6) Una solución se considerará inválida si selecciona capítulos no consecutivos de un mismo anime, si excede el tiempo disponible M o si excede la energía disponible E .
- (7) La fecha límite de entrega es el día **24 de junio de 2026**.

A. Casos de prueba

A.0.1. Generación de casos de prueba

Los estudiantes deberán implementar un generador de casos de prueba en Python llamado `testcases_generator.py`, ubicado en:

`code/implementation/scripts/testcases_generator.py`

Requisitos del generador:

- Debe generar archivos de entrada válidos para el problema **AniMarathon**.
- Cada archivo de entrada debe llamarse:

`testcases_{n}_{i}.txt`

donde:

- $\{n\}$ representa la cantidad de animes del caso de prueba.
- $\{i\}$ es un identificador único para diferenciar múltiples casos con la misma cantidad de animes.

- Formato del archivo de entrada:

- La primera línea contiene tres enteros n , M y E , donde:
 - n es la cantidad de animes disponibles.
 - M es la cantidad máxima de minutos disponibles.
 - E es la energía máxima disponible.
- Para cada anime i , se entrega una línea con:

`animei qi bi`

donde:

- `animei` es el nombre del anime, representado por un string sin espacios.
- q_i es la cantidad de capítulos del anime.
- b_i es el bono de satisfacción por completar todos los capítulos del anime.
- Luego, para cada uno de los q_i capítulos del anime i , se entrega una línea con:

`ti,j ci,j vi,j`

donde:

- $t_{i,j}$ es la duración del capítulo en minutos.
- $c_{i,j}$ es el costo de energía del capítulo.
- $v_{i,j}$ es la satisfacción obtenida al ver el capítulo.
- El generador debe permitir crear múltiples casos de prueba de manera automática, variando la cantidad de animes, cantidad de capítulos, minutos disponibles, energía disponible, puntajes de satisfacción y bonos de completación.

Los archivos generados serán utilizados por los algoritmos implementados en C++ para probar la corrección y eficiencia de las soluciones de fuerza bruta, greedy y programación dinámica.

A.0.2. Tipos de casos de prueba esperados

El generador debe ser capaz de producir, al menos, los siguientes tipos de casos:

1. **Casos pequeños:** utilizados para validar la correctitud del algoritmo de fuerza bruta y compararlo con programación dinámica. Se recomiendan casos con pocos animes y pocos capítulos, por ejemplo:

$$n \in \{3, 5, 8\}$$

con una cantidad total de capítulos pequeña.

2. **Casos medianos:** utilizados para comparar las heurísticas greedy con la solución óptima obtenida por programación dinámica. Se recomiendan valores como:

$$n \in \{20, 40, 80\}$$

3. **Casos grandes:** utilizados para medir rendimiento de los algoritmos eficientes. Se recomiendan valores como:

$$n \in \{100, 150, 200\}$$

En estos casos no se espera que fuerza bruta sea ejecutable en tiempos razonables.

Se recomienda usar el tipo de dato long long para prevenir errores de redondeo

A.0.3. Ejemplo de caso de prueba válido

El siguiente archivo corresponde a un caso de prueba válido para el problema:

```
4 240 90
shonen_quest 3 25
45 18 40
50 22 45
55 25 60
romcom_days 2 15
30 10 25
35 12 30
mecha_nova 2 50
60 30 65
70 35 75
slice_cafe 1 5
25 8 18
```

A.0.4. Validaciones esperadas

Los casos de prueba generados deben permitir verificar que:

- Los algoritmos no seleccionen capítulos aislados sin seleccionar los capítulos anteriores del mismo anime.
- Los algoritmos respeten el límite de minutos disponibles M .
- Los algoritmos respeten el límite de energía disponible E .
- Los bonos de completación se sumen únicamente cuando se seleccionan todos los capítulos de un anime.
- El algoritmo de fuerza bruta y el de programación dinámica entreguen el mismo resultado en casos pequeños.
- Las heurísticas greedy entreguen soluciones válidas, aunque no necesariamente óptimas.
- La programación dinámica pueda resolver casos medianos y grandes dentro de tiempos razonables.
- Los gráficos permitan observar diferencias claras entre los enfoques implementados.